

TSAR Project Assignment Part 3 - Group 3

Times Series Forecasting

Arpad Broglie

Teodor Glisic

2026

Table of contents

1	Preliminaries	2
1.1	Overview of tsibble	2
2	Model 1: Mean Method	3
2.1	Step 1: Extracting the Training Set	3
2.2	Step 2: Train 1st Benchmark Model	3
2.3	Step 3: Evaluate the Model Fit of 1st Benchmark Model	4
2.3.1	Task A	4
2.3.2	Task B	5
2.3.3	Task C	6
2.3.4	Task D	7
2.4	Step 4: Evaluate the Point Forecast Accuracy of your 1st Benchmark Model .	8
2.5	Step 5: Evaluate the Point Forecast Uncertainty of Your 1st Benchmark Model	10
3	Model 2: Seasonal naive method with drift	11
3.1	Step 1: Extracting the Training Set	11
3.2	Step 2: Train 2nd Benchmark Model	11
3.3	Step 3: Evaluate the Model Fit of Your 2nd Benchmark Model	12
3.3.1	Task A	12
3.3.2	Task B	13
3.3.3	Task C	14
3.3.4	Task D	15
3.4	Step 4: Evaluate the Point Forecast Accuracy of your 2nd Benchmark Model	16
3.5	Step 5: Evaluate the Point Forecast Uncertainty of Your 2nd Benchmark Model	18
4	Compare the Evaluation Results of the Two Models	19

1 Preliminaries

```
# Install needed libraries to use this report
if (!require(dplyr)) install.packages("dplyr")
if (!require(tidyr)) install.packages("tidyr")
if (!require(ggplot2)) install.packages("ggplot2")
if (!require(fpp3)) install.packages("fpp3")

#Load needed libraries
library(dplyr)
library(ggplot2)
library(tidyr)
library(fpp3)

# Read already filtered time series (tsibble)
# with Series ID CEU1000000001 (Mining and Logging)
mySeries <- readRDS("us_employment_filtered.rds")
```

1.1 Overview of tsibble

```
# Display the first few rows of the tsibble
head(mySeries)
```

```
# A tsibble: 6 x 4 [1M]
# Key:      Series_ID [1]
   Month Series_ID Title      Employed
   <mth> <chr>      <chr>      <dbl>
1 1939 Jan CEU1000000001 Mining and Logging 885
2 1939 Feb CEU1000000001 Mining and Logging 882
3 1939 Mär CEU1000000001 Mining and Logging 883
4 1939 Apr CEU1000000001 Mining and Logging 614
5 1939 Mai CEU1000000001 Mining and Logging 719
6 1939 Jun CEU1000000001 Mining and Logging 852
```

```
# Inspect the structure of the tsibble (index, keys, column data types)
str(mySeries)
```

```
tbl_ts [969 x 4] (S3: tbl_ts/tbl_df/tbl/data.frame)
 $ Month      : mth [1:969] 1939 Jan, 1939 Feb, 1939 Mär, 1939 Apr, 1939 Mai, 1939 Jun...
 $ Series_ID: chr [1:969] "CEU1000000001" "CEU1000000001" "CEU1000000001" "CEU1000000001"
```

```

$ Title      : chr [1:969] "Mining and Logging" "Mining and Logging" "Mining and Logging" "
$ Employed   : num [1:969] 885 882 883 614 719 852 847 864 894 941 ...
- attr(*, "key")= tibble [1 x 2] (S3: tbl_df/tbl/data.frame)
..$ Series_ID: chr "CEU1000000001"
..$ .rows     : list<int> [1:1]
.. ..$ : int [1:969] 1 2 3 4 5 6 7 8 9 10 ...
.. ..@ ptype: int(0)
..- attr(*, ".drop")= logi TRUE
- attr(*, "index")= chr "Month"
..- attr(*, "ordered")= logi TRUE
- attr(*, "index2")= chr "Month"
- attr(*, "interval")= interval [1:1] 1M
..@ .regular: logi TRUE

```

```

# Get a summary of the columns (distribution, range, missing values)
summary(mySeries)

```

Month	Series_ID	Title	Employed
Min. :1939 Jan	Length:969	Length:969	Min. : 556.0
1st Qu.:1959 Mär	Class :character	Class :character	1st Qu.: 676.0
Median :1979 Mai	Mode :character	Mode :character	Median : 765.0
Mean :1979 Mai			Mean : 793.8
3rd Qu.:1999 Jul			3rd Qu.: 899.0
Max. :2019 Sep			Max. :1260.0

2 Model 1: Mean Method

2.1 Step 1: Extracting the Training Set

```

# Extracting number of rows for training set with 80/20 split
overall_count <- (nrow(mySeries))
training_row_count <- floor((nrow(mySeries) * 0.8))
train <- mySeries %>% slice(1:training_row_count)

```

2.2 Step 2: Train 1st Benchmark Model

```

# Mean method used
mean_employment_fit <- train %>%
  model(
    Mean = MEAN(Employed)

```

```
)

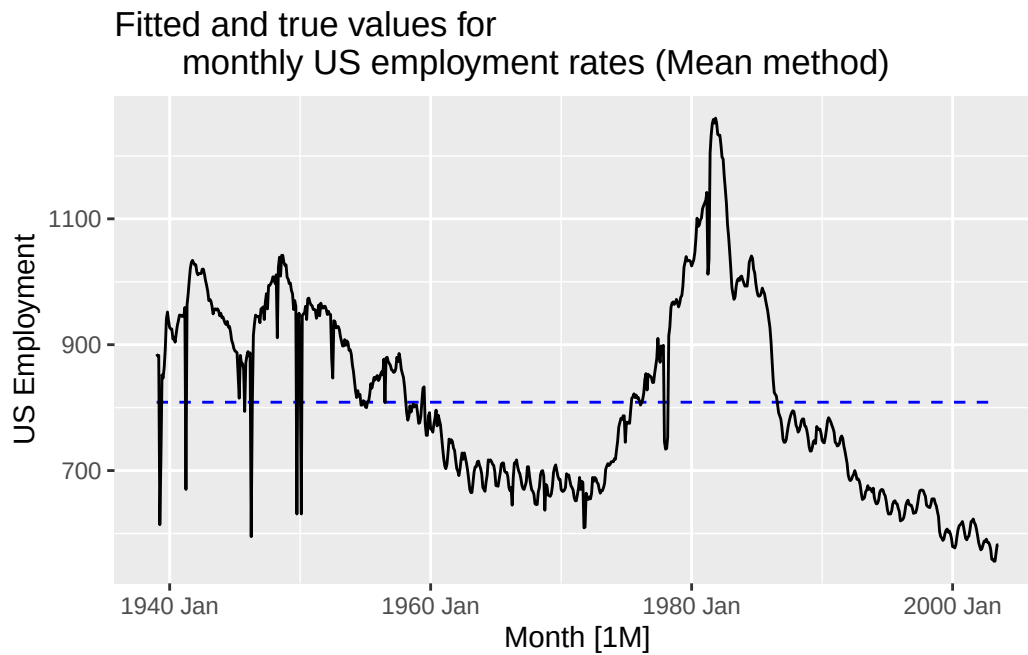
# View first impression of the fitted values
augment(mean_employment_fit)

# A tsibble: 775 x 7 [1M]
# Key:      Series_ID, .model [1]
  Series_ID      .model      Month Employed .fitted .resid .innov
  <chr>          <chr>      <mth>   <dbl>   <dbl>   <dbl>   <dbl>
1 CEU1000000001 Mean    1939 Jan      885    809.    76.4    76.4
2 CEU1000000001 Mean    1939 Feb      882    809.    73.4    73.4
3 CEU1000000001 Mean    1939 Mär      883    809.    74.4    74.4
4 CEU1000000001 Mean    1939 Apr      614    809.   -195.   -195.
5 CEU1000000001 Mean    1939 Mai      719    809.   -89.6   -89.6
6 CEU1000000001 Mean    1939 Jun      852    809.    43.4    43.4
7 CEU1000000001 Mean    1939 Jul      847    809.    38.4    38.4
8 CEU1000000001 Mean    1939 Aug      864    809.    55.4    55.4
9 CEU1000000001 Mean    1939 Sep      894    809.    85.4    85.4
10 CEU1000000001 Mean    1939 Okt      941    809.   132.    132.
# i 765 more rows
```

2.3 Step 3: Evaluate the Model Fit of 1st Benchmark Model

2.3.1 Task A

```
# Plot fitted values (predictions on training set)
# against actual values over time
augment(mean_employment_fit) %>%
  autoplot(.fitted, colour="blue", linetype="dashed") +
  autolayer(train, Employed) +
  labs(y= "US Employment",
       title="Fitted and true values for
monthly US employment rates (Mean method)"
  )
```



2.3.1.1 Answering questions to 1st Benchmark Model

Are the fitted values close to the actual values? No, the fitted values are not close to the actual values. They form a flat horizontal line at the historical average of the training data (approximately 808.6 thousand employees), completely failing to capture the cyclical, seasonal, and trending fluctuations of the true data.

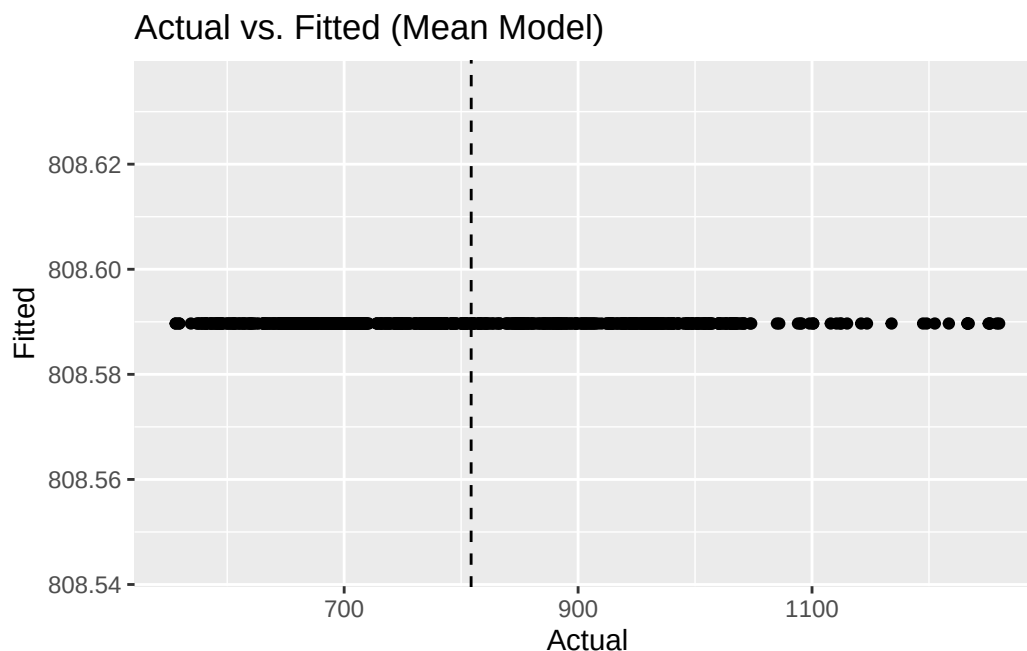
Are they systematically too high or too low? They are not systematically too high or too low overall (since the errors average out to zero over the training set), but they are systematically too low during cyclical peaks (e.g., the late 1970s and 1980s) and systematically too high during cyclical troughs (e.g., the 1940s and early 2000s).

If yes, where does it come from? (Consider how your benchmark model works!) This behavior is a direct result of how the Mean method operates: it calculates a single constant value (the arithmetic mean of the entire history) and projects it across all time periods, ignoring any temporal dependencies, trends, or seasonality.

2.3.2 Task B

```
# Create a scatter plot of actual values
# vs. fitted values with an identity line (slope=1)
augment(mean_employment_fit) %>%
  ggplot(aes(x=Employed, y=.fitted)) +
  geom_point() +
```

```
geom_abline(slope = 1, intercept = 0, linetype=2) +
labs(title = "Actual vs. Fitted (Mean Model)", x="Actual", y="Fitted")
```



2.3.2.1 Answering questions to 1st Benchmark Model

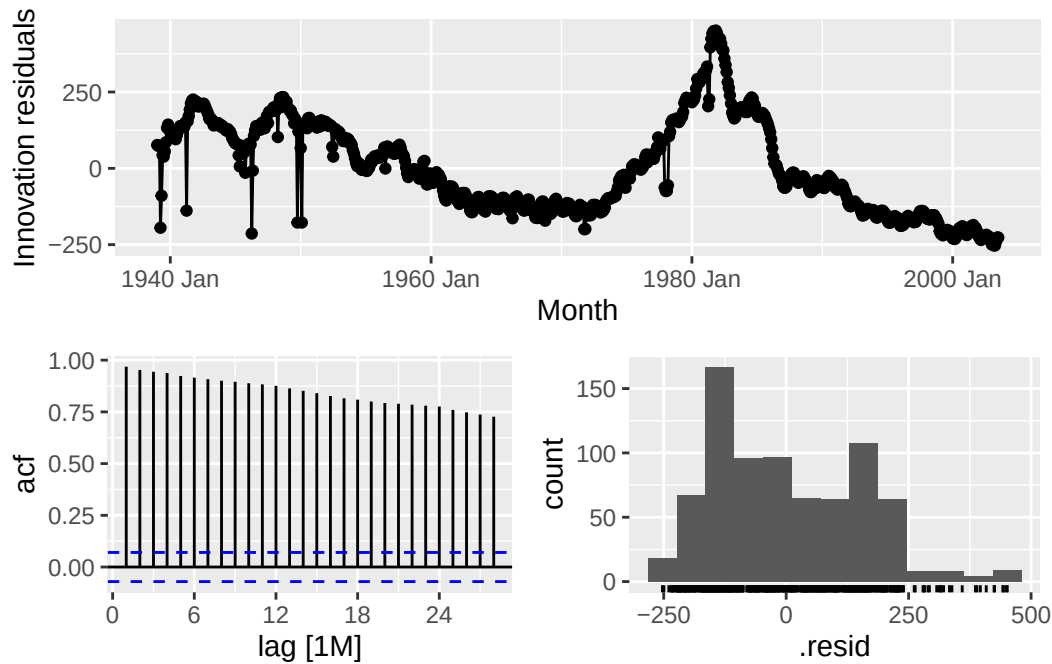
Are the points close to the identity line? No, the points do not align with the identity line. Instead, they form a vertical column of points because the fitted values are constant at the mean (808.6), while the actual values vary widely between 556 and 1260 thousand.

Are they systematically too high or too low? They are systematically too high for lower actual values (actual values below 808.6) and systematically too low for higher actual values (actual values above 808.6).

If yes, where does it come from? (Relate your answer to what you saw in the time plot.) This comes directly from the flat-line nature of the Mean model observed in the time plot: the model predicts the exact same average value regardless of whether the actual employment was historically at a cyclical low or a cyclical high.

2.3.3 Task C

```
# Generate diagnostic plots for the
# model residuals (time plot, ACF plot, histogram)
mean_employment_fit %>% gg_tsresiduals()
```



2.3.3.1 Answering questions to 1st Benchmark Model

Are the residuals auto-correlated? How do you decide that based on the plots? Yes, the residuals are highly auto-correlated. This is evident from the ACF plot of the residuals, which shows large, positive spikes at almost all lags that far exceed the blue dashed significance boundaries, as well as the strong, slow-moving cyclical waves visible in the residuals time plot.

Do the residuals have zero mean? How do you decide that based on the plots? Yes, the residuals have a mean of zero. This is visible in the residuals time plot, where the fluctuations are centered around the zero line, and in the residuals histogram, where the distribution is centered at zero.

What do these results tell you about your model? These results indicate that the Mean model is unbiased (zero mean), but it is extremely inadequate because it fails to capture any of the systematic temporal patterns (trends, cycles, and seasonality) present in the data, leaving all this predictable information in the residuals.

2.3.4 Task D

```
# Perform a Ljung-Box test for residual autocorrelation
# with 24 lags (2 * seasonal period)
augment(mean_employment_fit) %>%
  features(.innov, ljung_box, lag=24)
```

```
# A tibble: 1 x 4
  Series_ID      .model lb_stat lb_pvalue
  <chr>          <chr>   <dbl>   <dbl>
1 CEU1000000001 Mean     14183.     0
```

2.3.4.1 Answering questions to 1st Benchmark Model

Does the test result support your conclusions from (c)? Yes, the Ljung-Box test result strongly supports the conclusion that the residuals are auto-correlated.

How do you conclude that from the test result? The p-value of the Ljung-Box test is exactly 0 (which is less than the standard significance level of 0.05). This leads us to reject the null hypothesis (H_0 : residuals are white noise/uncorrelated) in favor of the alternative hypothesis (H_1 : residuals are autocorrelated).

```
# Calculate the mean of the residuals of the fitted model
residual_mean <- mean(
  augment(mean_employment_fit)$resid,
  na.rm = TRUE)
residual_mean
```

```
[1] -3.226362e-14
```

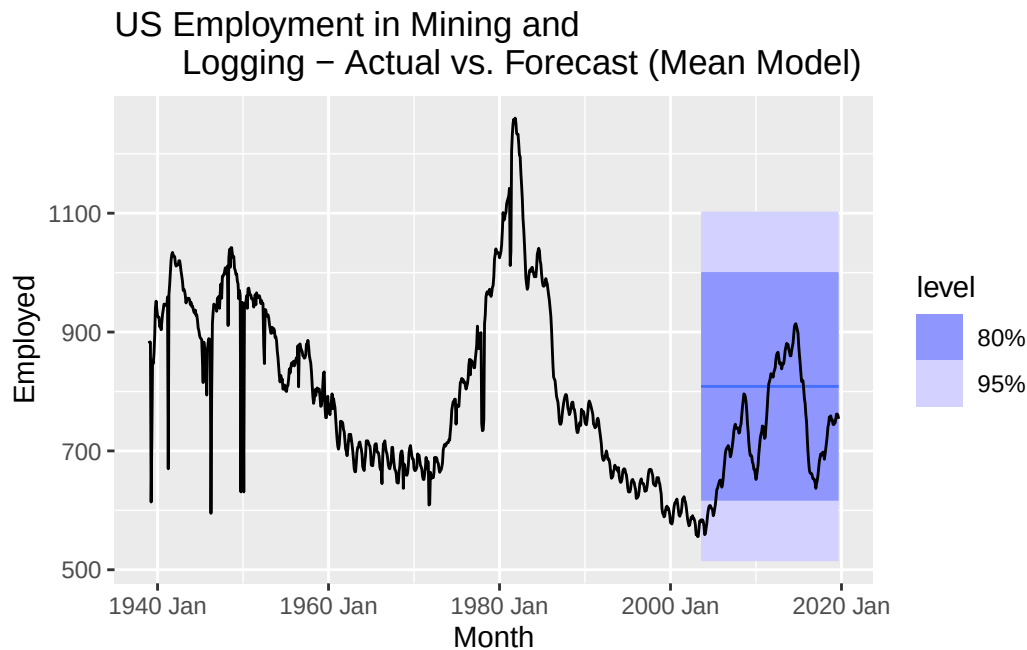
2.3.4.2 Answering questions to 1st Benchmark Model

Does the result support your conclusions from (c)? Yes, the calculated residual mean is approximately -3.23×10^{-14} (which is mathematically zero, subject to minor computer floating-point limitations), supporting the conclusion that the residuals have a zero mean and the model is unbiased on the training set.

2.4 Step 4: Evaluate the Point Forecast Accuracy of your 1st Benchmark Model

```
# Make a forecast and save the result
fc_mean <- mean_employment_fit %>%
  forecast(h = (nrow(mySeries) - nrow(train)))

# Create a time plot 'Actual vs. Forecast'
fc_mean %>%
  autoplot(mySeries) +
  labs(title="US Employment in Mining and
  Logging - Actual vs. Forecast (Mean Model)")
```

```
# Calculate the forecast accuracy metrics
fc_mean %>%
  accuracy(mySeries)
```

```
# A tibble: 1 x 11
  .model Series_ID .type ME RMSE MAE MPE MAPE MASE RMSSE ACF1
  <chr> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mean CEU1000000001 Test -74.0 116. 98.8 -11.7 14.6 2.27 1.70 0.987
```

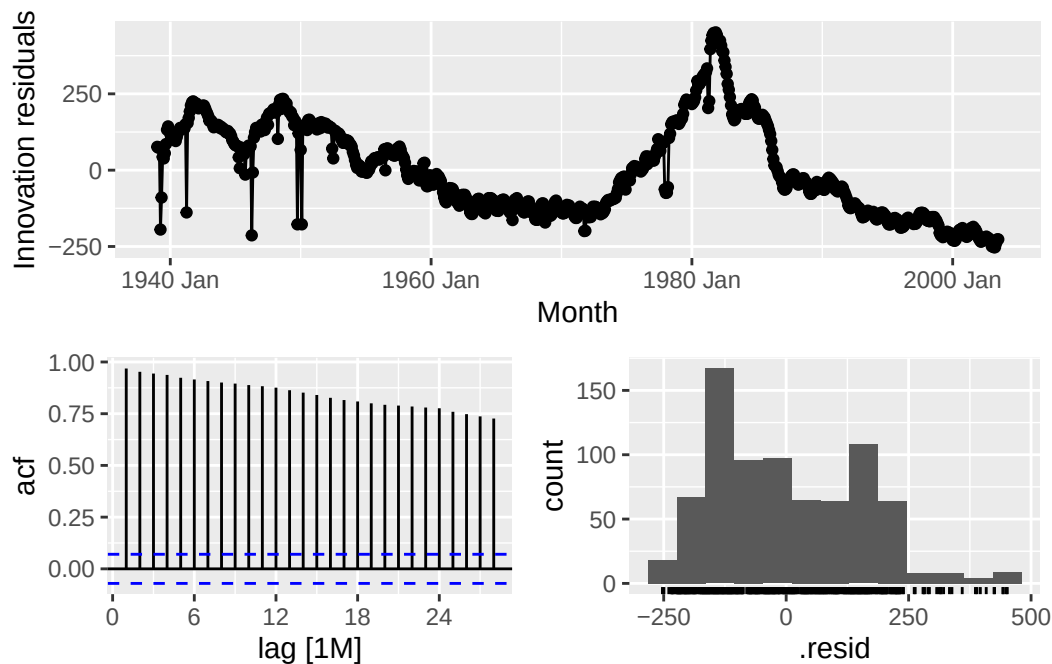
2.4.0.1 Answering questions to 1st Benchmark Model

*Interpret the **RMSE** (Root Mean Squared Error) in max. 3 sentences:* The RMSE on the test set is 116.3, which represents the standard deviation of the forecast errors. Since employment is measured in thousands, this means the model's point forecasts deviate from the true values by an average of 116.3 thousand employees on the test set. Because the RMSE is scale-dependent, it must be evaluated by a domain expert relative to the overall employment level (ranging between 584 and 754 thousand in the test set), indicating a moderately large prediction error.

*Interpret the **MAPE** (Mean Absolute Percentage Error) in max. 3 sentences:* The MAPE on the test set is 14.6%, which measures the average size of the forecast errors relative to the actual levels as a percentage. This means that, on average, the Mean model's point forecasts are off by 14.6% of the true employment level. This scale-independent metric confirms that the model's accuracy is relatively poor, as a 14.6% average error is quite high for macro-employment data.

2.5 Step 5: Evaluate the Point Forecast Uncertainty of Your 1st Benchmark Model

```
# Residual diagnostics for checking homoscedasticity and normality
mean_employment_fit %>% gg_tsresiduals()
```



```
# Display prediction interval boundaries
fc_mean %>%
  hilo()
```

```
# A tsibble: 194 x 7 [1M]
# Key:   Series_ID, .model [1]
  Series_ID      .model    Month
  <chr>          <chr>    <mth>
1 CEU1000000001 Mean     2003 Aug
2 CEU1000000001 Mean     2003 Sep
3 CEU1000000001 Mean     2003 Okt
4 CEU1000000001 Mean     2003 Nov
5 CEU1000000001 Mean     2003 Dez
6 CEU1000000001 Mean     2004 Jan
7 CEU1000000001 Mean     2004 Feb
8 CEU1000000001 Mean     2004 Mär
9 CEU1000000001 Mean     2004 Apr
10 CEU1000000001 Mean     2004 Mai
```

```
# i 184 more rows
# i 4 more variables: Employed <dist>, .mean <dbl>, `80%` <hilo>, `95%` <hilo>
```

2.5.0.1 Answering questions to 1st Benchmark Model

Are the residuals homoscedastic? (That is, do they have constant variance?) No, the residuals are heteroscedastic. The residual time plot shows that the variance (vertical spread) of the residuals is not constant, with much larger fluctuations in the middle of the training set (1970s–1990s) than at the beginning (1940s) or end.

Are the residuals normally distributed? No, the residuals are not normally distributed. The histogram shows a multi-modal, non-symmetric distribution with multiple peaks that does not resemble a classic bell curve.

Does it make sense to include these prediction intervals in your model evaluation? Why/why not? No, it does not make sense to include these prediction intervals. The analytical calculation of prediction intervals relies on the assumptions that the residuals are uncorrelated, homoscedastic, and normally distributed; since all three assumptions are severely violated here, the intervals are distorted and highly unreliable.

3 Model 2: Seasonal naive method with drift

3.1 Step 1: Extracting the Training Set

```
# Extracting number of rows for training set with 80/20 split
overall_count <- (nrow(mySeries))
training_row_count <- floor((nrow(mySeries) * 0.8))
train <- mySeries %>% slice(1:training_row_count)
```

3.2 Step 2: Train 2nd Benchmark Model

```
# Train the model with the seasonal naive drift method
naive_drift_employment_fit <- train %>% model (
  SNaive_Drift = SNAIVE(Employed ~ drift())
)

# View the trained data
augment(naive_drift_employment_fit)
```

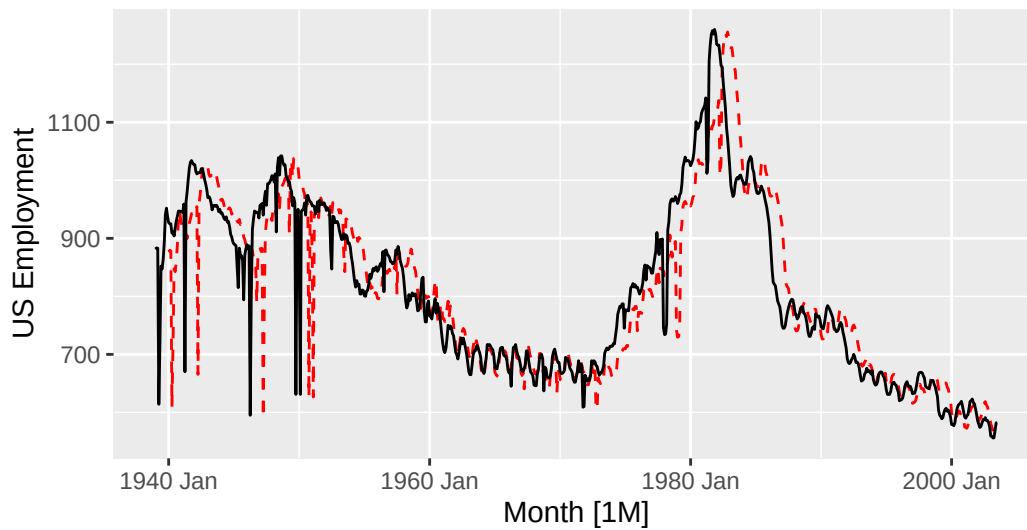
```
# A tsibble: 775 x 7 [1M]
# Key:      Series_ID, .model [1]
  Series_ID      .model      Month Employed .fitted .resid .innov
  <chr>          <chr>      <mt>    <dbl>    <dbl>    <dbl>    <dbl>
1 CEU1000000001 SNaive_Drift 1939 Jan      885      NA      NA      NA
2 CEU1000000001 SNaive_Drift 1939 Feb      882      NA      NA      NA
3 CEU1000000001 SNaive_Drift 1939 Mär      883      NA      NA      NA
4 CEU1000000001 SNaive_Drift 1939 Apr      614      NA      NA      NA
5 CEU1000000001 SNaive_Drift 1939 Mai      719      NA      NA      NA
6 CEU1000000001 SNaive_Drift 1939 Jun      852      NA      NA      NA
7 CEU1000000001 SNaive_Drift 1939 Jul      847      NA      NA      NA
8 CEU1000000001 SNaive_Drift 1939 Aug      864      NA      NA      NA
9 CEU1000000001 SNaive_Drift 1939 Sep      894      NA      NA      NA
10 CEU1000000001 SNaive_Drift 1939 Okt      941      NA      NA      NA
# i 765 more rows
```

3.3 Step 3: Evaluate the Model Fit of Your 2nd Benchmark Model

3.3.1 Task A

```
# Plot actual training data against the model's fitted values over time
augment(naive_drift_employment_fit) %>%
  autoplot(.fitted, colour="red", linetype="dashed") +
  autolayer(train, Employed) +
  labs(y= "US Employment",
       title="Fitted and true values for
monthly US employment rates
(Seasonal Naive with Drift method)"
  )
```

Fitted and true values for
monthly US employment rates
(Seasonal Naive with Drift method)



3.3.1.1 Answering questions to 2nd Benchmark Model

Are the fitted values close to the actual values? Yes, the fitted values are much closer to the actual values compared to the Mean model. They successfully track the strong seasonal monthly fluctuations and the long-term downward trend present in the training set.

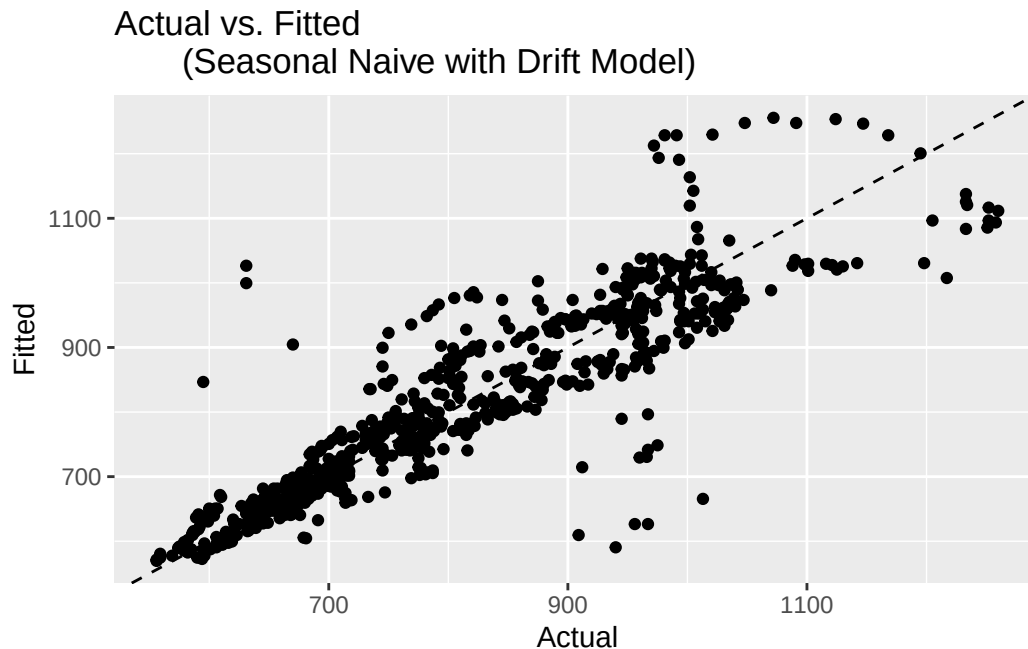
Are they systematically too high or too low? They are not systematically too high or too low overall, but they lag behind by one year (12 months) during sudden turns or trend changes in the data.

If yes, where does it come from? (Consider how your benchmark model works!) This close tracking comes from the design of the Seasonal Naïve with Drift model: it sets the 1-step fitted value to the value from the same month of the previous year plus the average monthly trend (drift). The slight lag at turning points happens because the model relies entirely on historical values from 12 months ago to predict today's value.

3.3.2 Task B

```
# Plot actual vs. fitted values to check deviation from the identity line (slope=1)
augment(naive_drift_employment_fit) %>%
  ggplot(aes(x=Employed, y=.fitted)) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0, linetype=2) +
  labs(title = "Actual vs. Fitted
           (Seasonal Naive with Drift Model)",
```

```
x="Actual",
y="Fitted"
)
```



3.3.2.1 Answering questions to 2nd Benchmark Model

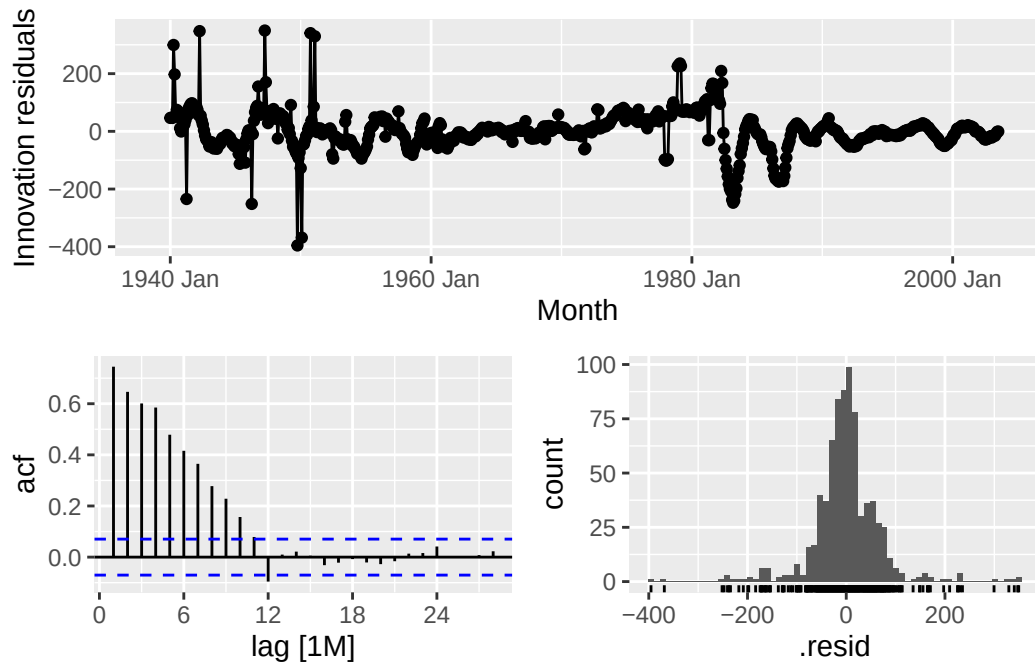
Are the points close to the identity line? Yes, the points are clustered relatively closely along the dashed identity line ($y = x$), showing a massive improvement in fit over the Mean model.

Are they systematically too high or too low? There is no significant systematic bias, as the points are distributed fairly evenly on both sides of the identity line.

If yes, where does it come from? (Relate your answer to what you saw in the time plot.) This tight alignment around the identity line reflects the model's ability to successfully capture the seasonal level and trend of the training series over time, keeping the fitted values close to the actual values.

3.3.3 Task C

```
# View residual diagnostic plots (time plot, ACF plot, histogram)
# to assess model fit
naive_drift_employment_fit %>% gg_tsresiduals()
```



3.3.3.1 Answering questions to 2nd Benchmark Model

Are the residuals auto-correlated? How do you decide that based on the plots? Yes, the residuals are still auto-correlated. Although much smaller than in the Mean model, the ACF plot shows significant positive spikes at lag 1 and lag 12 (as well as multiples of 12), showing that some short-term and seasonal dependencies have not been fully captured.

Do the residuals have zero mean? How do you decide that based on the plots? Yes, the residuals have a mean of zero. The residuals time plot is centered around the zero line, and the histogram distribution peaks exactly at zero.

What do these results tell you about your model? These results tell us that while the Seasonal Naïve with Drift model is unbiased and fits the training data well, it is still not a perfect model because it leaves significant autocorrelation in the residuals, indicating that a more advanced model (like ETS or ARIMA) could capture the remaining patterns.

3.3.4 Task D

```
# Conduct a formal Ljung-Box test for autocorrelation
# in the residuals (lag = 2 * seasonal period)
augment(naive_drift_employment_fit) %>%
  features(
    .innov,
    ljung_box,
```

```
lag = 24
)
```

```
# A tibble: 1 x 4
  Series_ID      .model      lb_stat lb_pvalue
  <chr>          <chr>      <dbl>   <dbl>
1 CEU1000000001 SNaive_Drift 1834.     0
```

3.3.4.1 Answering questions to 2nd Benchmark Model

Does the test result support your conclusions from (c)? Yes, the Ljung-Box test result supports the conclusion that the residuals are auto-correlated.

How do you conclude that from the test result? The p-value of the test is exactly 0 (which is less than 0.05). This leads us to reject the null hypothesis of white noise and conclude that there is statistically significant autocorrelation left in the residuals.

```
# Calculate the mean of the residuals to check for forecast bias
residual_mean_naive_drift <- mean(
  augment(naive_drift_employment_fit)$resid,
  na.rm = TRUE)
residual_mean_naive_drift
```

```
[1] -4.170354e-14
```

3.3.4.2 Answering questions to 2nd Benchmark Model

Does the result support your conclusions from (c)? Yes, the calculated residual mean is approximately -4.17×10^{-14} (which is practically zero), supporting the conclusion that the model is unbiased on the training set.

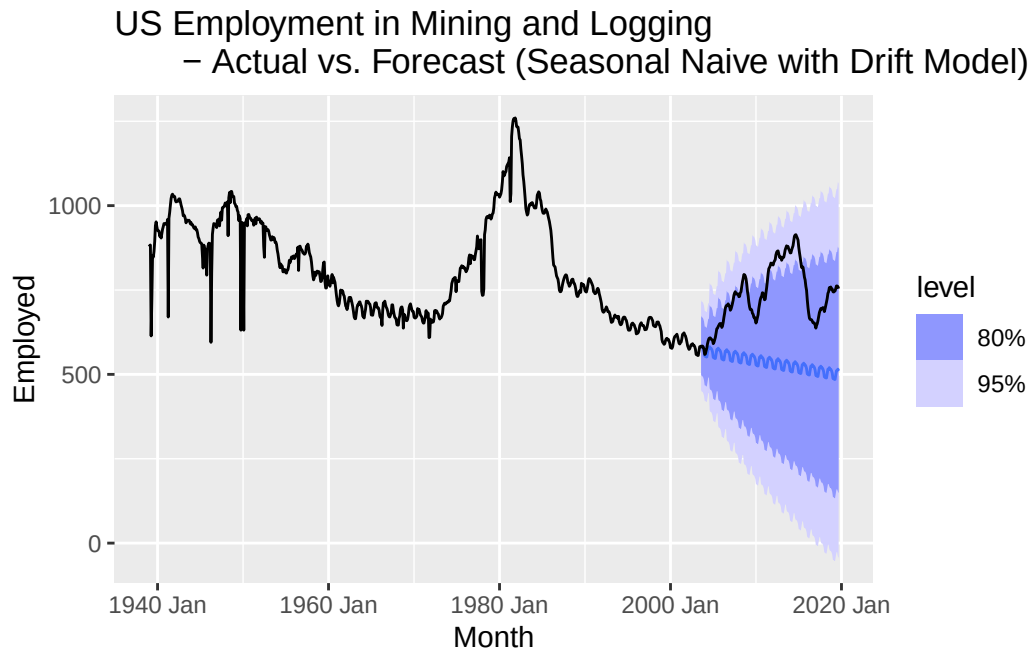
3.4 Step 4: Evaluate the Point Forecast Accuracy of your 2nd Benchmark Model

```
# Generate forecasts for the test set period
fc_naive_drift <- naive_drift_employment_fit %>%
  forecast(h=(overall_count - training_row_count))

# Plot the forecast values with prediction
# intervals against the actual complete dataset
fc_naive_drift %>%
  autoplot(mySeries) +
```



```
labs(title="US Employment in Mining and Logging
- Actual vs. Forecast (Seasonal Naive with Drift Model)"
)
```



```
# Calculate point forecast accuracy metrics
# (MAE, RMSE, MAPE, MASE) on the test set
fc_naive_drift %>% accuracy(mySeries)
```

```
# A tibble: 1 x 11
  .model      Series_ID .type    ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 SNaive_Drift CEU1000000~ Test   199.  223.  199.  25.8  25.9  4.56  3.27  0.986
```

3.4.0.1 Answering questions to 2nd Benchmark Model

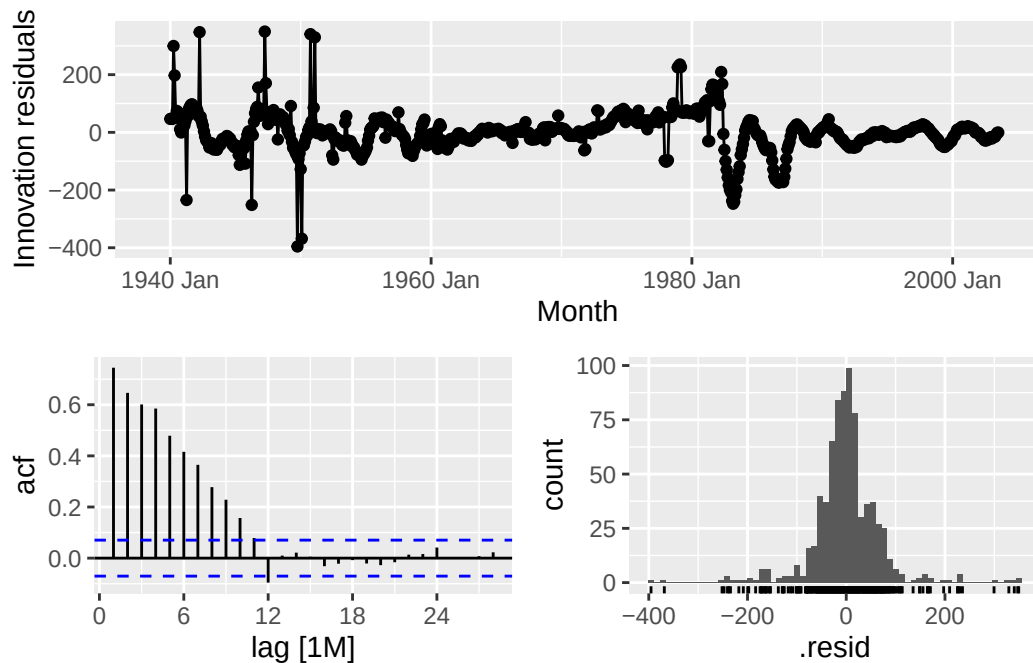
Interpret the **RMSE** (Root Mean Squared Error) in max. 3 sentences: The RMSE on the test set is 223.2, which means the standard deviation of the forecast errors is 223.2 thousand employees. This is nearly double the RMSE of the Mean model (116.3), indicating very poor point forecast performance on the test set. This large error is caused by the model projecting a continuous downward trend while the actual test data turned and trended upwards.

Interpret the **MAPE** (Mean Absolute Percentage Error) in max. 3 sentences: The MAPE on the test set is 25.9%, which means that the model's forecasts are, on average, off by 25.9% of the actual employment level. This high percentage error shows that the model performs poorly on the test set, failing to compete with the simpler Mean model (which has a MAPE

of 14.6%). This scale-independent metric highlights the risk of extrapolating a historical trend that does not persist into the future.

3.5 Step 5: Evaluate the Point Forecast Uncertainty of Your 2nd Benchmark Model

```
# Perform residual diagnostics to assess homoscedasticity and normality
naive_drift_employment_fit %>% gg_tsresiduals()
```



```
# Extract the upper and lower boundaries for
# 80% and 95% prediction intervals
fc_naive_drift %>% hilo()
```

```
# A tsibble: 194 x 7 [1M]
# Key:   Series_ID, .model [1]
  Series_ID      .model      Month
  <chr>          <chr>      <mth>
1 CEU1000000001 SNaive_Drift 2003 Aug
2 CEU1000000001 SNaive_Drift 2003 Sep
3 CEU1000000001 SNaive_Drift 2003 Okt
4 CEU1000000001 SNaive_Drift 2003 Nov
5 CEU1000000001 SNaive_Drift 2003 Dez
6 CEU1000000001 SNaive_Drift 2004 Jan
```

```

7 CEU1000000001 SNaive_Drift 2004 Feb
8 CEU1000000001 SNaive_Drift 2004 Mär
9 CEU1000000001 SNaive_Drift 2004 Apr
10 CEU1000000001 SNaive_Drift 2004 Mai
# i 184 more rows
# i 4 more variables: Employed <dist>, .mean <dbl>, `80%` <hilo>, `95%` <hilo>

```

3.5.0.1 Answering questions to 2nd Benchmark Model

Are the residuals homoscedastic? (That is, do they have constant variance?) No, the residuals are heteroscedastic. The residual time plot shows that the variance changes across the training set, with fluctuations being noticeably wider in the middle of the series than at the start.

Are the residuals normally distributed? No, the residuals are not normally distributed. The histogram is asymmetric, skewed, and exhibits multiple peaks, failing to show a normal bell shape.

Does it make sense to include these prediction intervals in your model evaluation? Why/why not? No, it does not make sense. Because the residuals violate the assumptions of normality, homoscedasticity, and independence, the calculated prediction intervals are highly unreliable and likely distorted.

4 Compare the Evaluation Results of the Two Models

Although the Seasonal Naïve with Drift model provides a dramatically better fit on the training data (as it successfully tracks seasonal cycles and the training trend), it performs significantly worse on the test set than the simple Mean model. The Mean model achieves a test RMSE of 116.3 and a MAPE of 14.6%, whereas the Seasonal Naïve with Drift model has an RMSE of 223.2 and a MAPE of 25.9%. This discrepancy is due to a structural break: the training set (Jan 1939 to Jul 2003) exhibits a long-term downward trend, which the drift model projects into the future, whereas the actual test set (Aug 2003 to Sep 2019) reverses and trends upward. Therefore, the Mean model is the better forecasting model for this specific test set, though neither model is ideal since both leave significant autocorrelation in their residuals.